

Method: TimeRCD(Stage1) + MLLM 검증/경계보정(Stage2) 파일럿

목적: SMD 다변량 이상탐지에서, 자체 Stage1(DINOv2 patch-KNN) 대신 zero-shot 파운데이션 모델 TimeRCD를 Stage1(위치탐지)로 쓰고, 기존에 검증된 Stage2 MLLM 파이프라인(단변량 NAB/SMAP/MSL에서 F1 0.6174->0.679 개선 실적)을 그대로 재사용해 SMD 5개 held-out entity(machine-2-2, 2-9, 3-1, 3-2, 3-8)에 적용한다.

핵심 제약: TimeRCD, Stage2 MLLM 둘 다 SMD train 데이터로 fine-tuning하지 않는다(zero-shot 유지). Stage1(자체 patch-KNN) 코드는 삭제하지 않고 그대로 둔다(추후 비교/양상불용).

전체 흐름:

- [1] TimeRCD로 각 entity의 시점별(point-wise) anomaly score 생성 (zero-shot, 이미 사전학습된 가중치만 사용, 2.5B synthetic points로 원저자가 학습한 것)
- [2] 그 점수를 percentile threshold + merge로 '느슨한' 후보 구간(candidate interval) 리스트로 변환 -- 재현율 위주, 정밀도는 다음 단계에서 개선
- [3] 후보마다 경계 후보 지점(boundary option) 계산 + 수치 요약(numerical summary) 생성 + 시각화 이미지 렌더링
- [4] 후보를 v5.2c 모드 선택 규칙에 따라 '보호(A)' 또는 '검증대상(C)'으로 분류
- [5] GPT-4o에 이미지+수치요약을 보내 (검증대상만) keep/discard/uncertain 판정, 그리고 (모든 생존 후보) 최적 경계(L/R option) 선택을 요청
- [6] 최종 채택된 구간으로 interval F1 / point-wise F1 / PA F1(point-adjustment) 재계산, TimeRCD 단독 결과와 비교

Stage 1 -- TimeRCD 후보 생성 (알고리즘)

입력: TimeRCD score{0..T-1} (시점별 0~1 anomaly probability, TimeRCD 공식 추론 결과)

- 1) 임계값: $\text{thr} = \text{percentile}(\text{score}, 100 - \text{LOOSE_PCT})$, $\text{LOOSE_PCT} = 10.0$
-> 상위 10%에 해당하는 값 이상인 시점만 1차 후보로 표시
- 2) 이진화: $\text{binary}(t) = 1 \text{ if } \text{score}(t) \geq \text{thr} \text{ else } 0$
- 3) 연속 구간 추출 $\text{get_intervals}(\text{binary})$: binary가 1로 이어지는 [start, end] (양끝 포함) 구간들
- 4) 병합: 인접한 두 구간의 간격이 $\text{MERGE_GAP}=100 \text{ tick}$ 이하이면 하나로 합침
 $\text{merged}[-1] = (\text{merged}[-1][0], \text{iv}[1]) \text{ if } \text{iv}[0] - \text{merged}[-1][1] \leq 100$
- 5) 필터: 길이가 $\text{MIN_IV}=10 \text{ tick}$ 미만인 구간은 노이즈로 간주해 폐기

결과: 후보 interval 리스트 $\text{candidates} = \{(s_0, e_0), (s_1, e_1), \dots\}$

설계 의도: 원본 Stage2(univariate)의 stage1() 함수와 동일한 '느슨한 후보 생성' 철학을 유지.
재현율을 100%에 가깝게 두고, 정밀도 개선은 전적으로 Stage2(MLLM)에 위임한다.

실측 예시 (본 파일럿 실행 결과):

machine-2-2: T=23700, 후보 48개, GT 세그먼트 11개, Stage1(느슨한 후보) interval F1=0.400
(recall=100%, precision=25% -- 설계대로 재현율 위주)

machine-2-9: T=28722, 후보 44개, GT 세그먼트 10개

machine-3-1: T=28700, 후보 42개, GT 세그먼트 4개

machine-3-2: T=23703, 후보 47개, GT 세그먼트 10개

machine-3-8: T=28704, 후보 47개, GT 세그먼트 6개

경계 후보(boundary option) 생성 알고리즘

목적: Stage1이 준 후보 [s0,e0]이 실제 이상의 시작/끝과 정확히 일치하지 않을 수 있으므로,
'그럴듯한 대안 경계 지점' 몇 개를 통계적으로 미리 계산해 VLM에게 '이 중에서 골라라'고 제시.
VLM이 임의의 숫자를 만들어내지 않고, 미리 계산된 후보 중 선택만 하도록 제한(hallucination 방지).

$L = e0 - s0 + 1$ (후보 길이), $\text{margin} = \max(3 * L, 100)$
 $\text{smooth} = \text{uniform_filter1d}(\text{score}, \text{size}=\max(5, L//4))$ <- 이동평균으로 국소 스무딩

구간 내부 통계: $\text{inner} = \text{smooth}[s0:e0+1]$
 $\text{tau}_h = \text{percentile}(\text{inner}, 50)$ (중간값 -- '절반 정도 내려간 지점' 기준)
 $\text{tau}_l = \text{percentile}(\text{inner}, 25)$ ('많이 내려간 지점' 기준)

왼쪽 경계 후보 (margin 범위 내에서 s0 기준 왼쪽으로 탐색):

L0 'original' = s0 그대로
L1 'high_cross' = s0에서 왼쪽으로 가다가 처음 $\text{smooth} < \text{tau}_h$ 가 되는 지점
L2 'low_cross' = " 처음 $\text{smooth} < \text{tau}_l$ 가 되는 지점
L3 'local_min' = margin 구간 내 극소점(scipy.signal.argrelemin) 중 s0에 가장 가까운 것
L4 'deriv_rise' = margin 구간 내 1차 미분(np.diff)이 가장 큰(급상승) 지점

오른쪽 경계 후보도 대칭적으로 R0 original / R1 high_cross / R2 low_cross / R3 local_min /
R4 deriv_fall(가장 급하게 떨어지는 지점) 계산. 중복 지점은 제거(dedup).

수치 요약 make_summary(): 후보 앞(pre)/안(inside)/뒤(post) 구간의 평균·표준편차,
피크 점수(peak_score), 평균 점수(mean_score), 전체 대비 백분위(score_pct)를 계산해
이미지와 함께 텍스트로도 VLM에게 제공.

후보별 처리 모드 선택 (v5.2c, Selective Verification)

n_cands = 이 entity의 전체 후보 개수, top1_idx = peak_score가 가장 높은 후보의 인덱스

```
if n_cands <= 2:
    모든 후보 -> Mode A (보호, Protected)
else:
    cid == top1_idx -> Mode A (보호)
    그 외 나머지 -> Mode C (검증대상, Verify+Boundary)
```

Mode A (보호, boundary-only):

‘이미 이상으로 확정됐다’고 가정하고 keep/discard 질문을 생략, 곧바로 경계선택 질문 1회만 호출.
이유: 가장 확신도 높은 후보(top1)까지 검증 질문에 노출시키면, VLM이 지나치게 보수적으로 굴 때 진짜 양성(true positive)을 discard해버리는 리스크가 있음 -> 이를 원천 차단.

Mode C (검증대상, Verify -> Boundary):

- 1) call_verify: keep / discard / uncertain 3지선다
- 2) discard가 아니면(keep 또는 uncertain) -> call_boundary 호출해서 경계까지 선택
discard된 후보는 최종 kept_ivs에서 완전히 제외됨.

이번 파일럿 실측(machine-2-2, 후보 48개): 보호(A) 1개, 검증대상(C) 47개.

결과: kept=38/48 (10개 discard), tp_retention=100%(진짜 양성은 하나도 안 버림),
fp_rejection=28%(가짜 후보의 28%를 걸러냄).

VLM 프롬프트 원문 -- (1) 검증(Verify) 호출

시스템 프롬프트 (SYS_VERIFY):

"You are a false-positive rejection filter for a time-series anomaly detector. The candidate was proposed by a high-recall visual detector. Your role is ONLY to reject clear false positives. When in doubt, choose 'uncertain'. Do not be aggressive in discarding."

유저 프롬프트 (prompt_verify, 실제 값 예시로 채움):

Domain: SMD industrial server telemetry (TimeRCD zero-shot anomaly score, combined over 38 channels)

Candidate #<cid>: [<s0>, <e0>]

=== NUMERICAL SUMMARY ===

interval, length, peak_score, mean_score, score_pct,
pre_mean, pre_std, inside_mean, inside_std, post_mean, post_std

Images show: [1] global series with candidate highlighted (red),
[2] local zoom around the candidate.

=== YOUR TASK ===

Decide whether this candidate is:

keep -- shows a genuine anomaly (spike, level shift, trend change)
discard -- clearly a normal fluctuation consistent with the rest of the series
uncertain -- ambiguous, domain-specific, or cannot be confidently rejected

IMPORTANT: discard ONLY if the evidence strongly shows normal behavior. If the candidate shows ANY unusual pattern, unusual magnitude, or domain-specific signal, choose 'keep' or 'uncertain'.

Return ONLY valid JSON: {"decision": "keep"}

Valid values: "keep", "discard", "uncertain"

첨부 이미지: 2장 (make_images_verify 함수로 생성, 다음 페이지에 실제 예시)

VLM 프롬프트 원문 -- (2) 경계보정(Boundary) 호출

시스템 프롬프트 (SYS_BOUNDARY):

"You are a time-series anomaly detection assistant. The candidate interval is already confirmed as anomalous. Your task is ONLY to select the best left and right temporal boundary from the pre-computed options. Do not invent coordinates outside the list."

유저 프롬프트 (prompt_boundary, 실제 값 예시로 채움):

Domain: SMD industrial server telemetry ...

Candidate #<cid>: [{<s0>, <e0>}]

=== LEFT BOUNDARY OPTIONS ===

L0: original = index <s0>

L1: high_cross = index <t> (있을 경우)

L2: low_cross = index <t> (있을 경우)

L3: local_min = index <t> (있을 경우)

L4: deriv_rise = index <t> (있을 경우)

=== RIGHT BOUNDARY OPTIONS === (R0~R4, 대칭 구조)

=== NUMERICAL SUMMARY === (검증 호출과 동일한 필드)

The candidate is anomalous. Select which boundary options best capture its true extent.

L0 and R0 are the Stage-1 boundaries. Other options may extend the boundary.

Images show: [1] global series [2] local zoom [3] TimeRCD score curve.

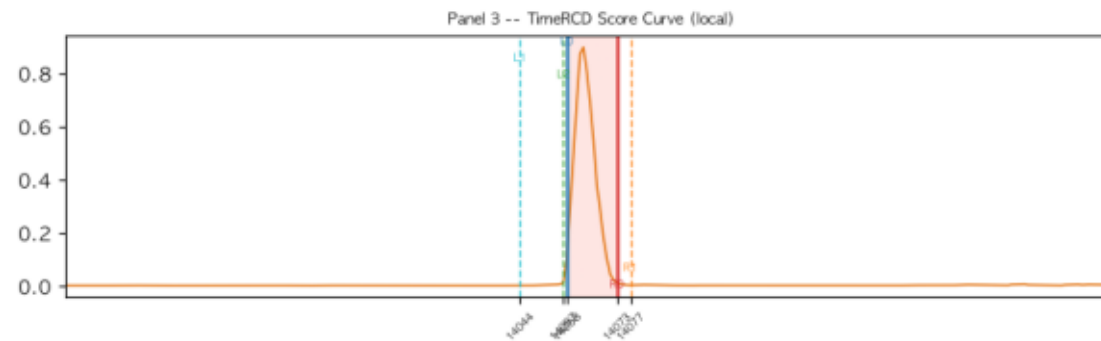
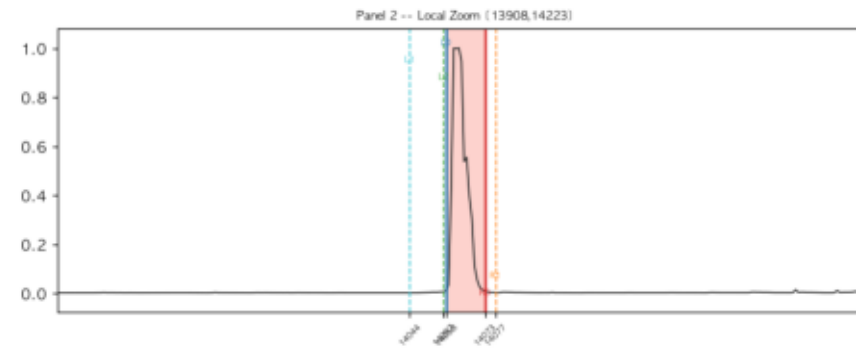
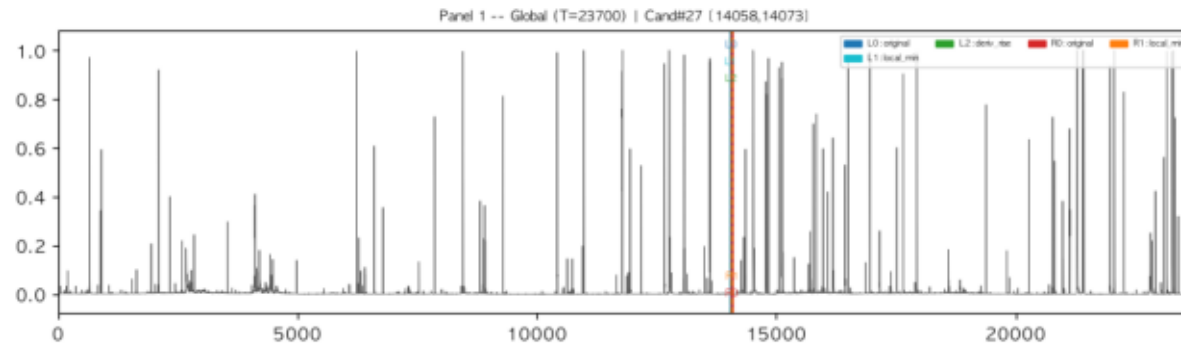
Boundary options are labeled L0-L{n-1} (blue) and R0-R{n-1} (red).

Return ONLY valid JSON: {"left_option": 0, "right_option": 0}

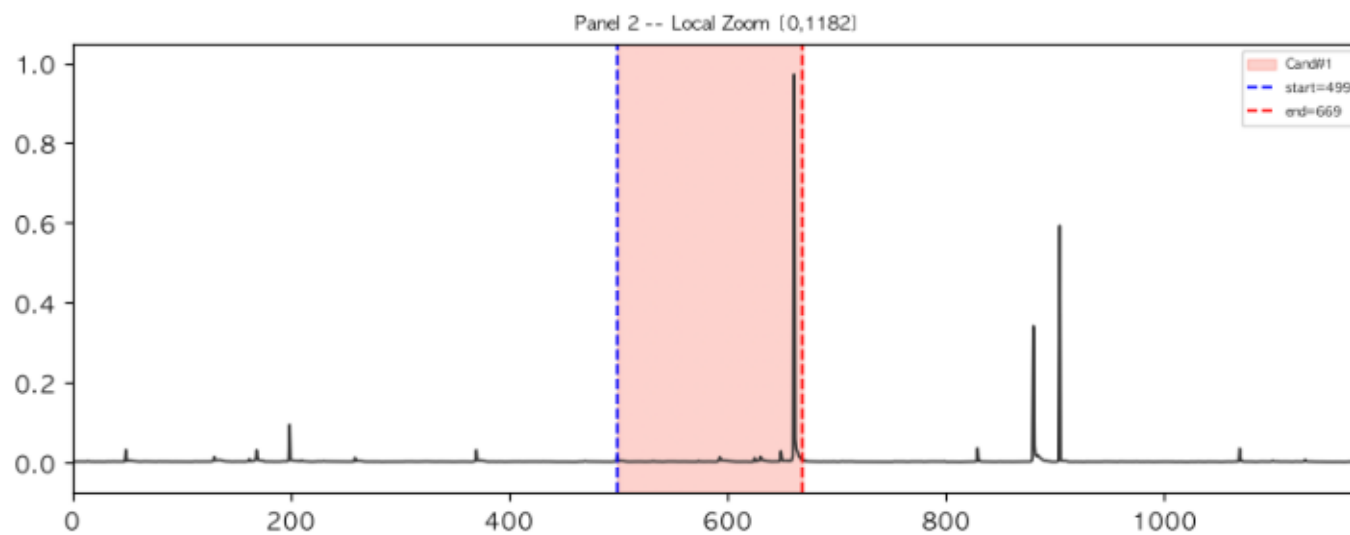
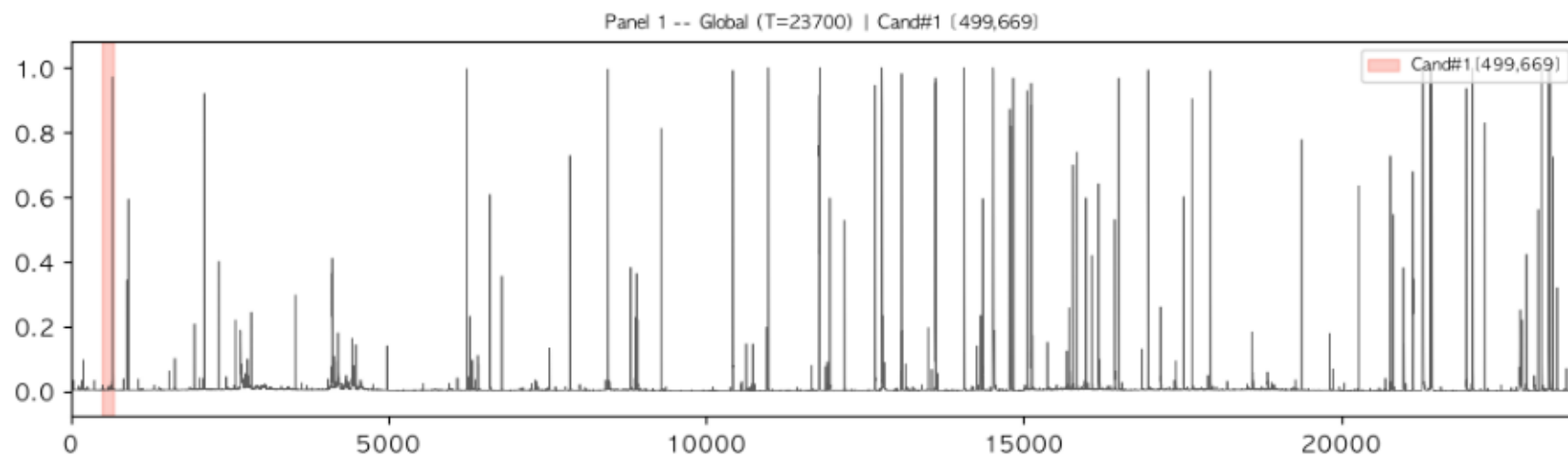
Integers in range [0, n_left-1] and [0, n_right-1].

첨부 이미지: 3장 (make_images_with_opts 함수로 생성, 다음 페이지에 실제 예시)

실제 예시 -- machine-2-2 Cand#27(보호/Mode A) [14058,14073] 에 보내는 경계보정 이미지 3장



실제 예시 -- machine-2-2 Cand#1 (검증대상/Mode C) [499,669] 에 보내는 검증용 이미지 2장



수치 요약(numerical summary) 실측 예시

Cand#27 (보호모드, machine-2-2) [14058,14073]:

```
{"interval": [14058, 14073], "length": 16, "peak_score": 0.8963, "mean_score": 0.4005, "score_pct": 99.9,
"pre_mean": 0.0013, "pre_std": 0.0008, "inside_mean": 0.4012, "inside_std": 0.384, "post_mean": 0.0016,
"post_std": 0.0008}
```

Cand#1 (검증대상, machine-2-2) [499,669]:

```
{"interval": [499, 669], "length": 171, "peak_score": 0.0315, "mean_score": 0.0076, "score_pct": 94.5,
"pre_mean": 0.0029, "pre_std": 0.0048, "inside_mean": 0.0098, "inside_std": 0.074, "post_mean": 0.0043,
"post_std": 0.0302}
```

필드 설명: interval=[시작,끝], length=길이(tick), peak_score=구간내 스무딩점수 최댓값,
mean_score=구간내 스무딩점수 평균, score_pct=전체 시계열 대비 백분위(피크 기준),
pre/inside/post_mean,std = 후보 앞/안/뒤 구간(margin=max(3*L,100))의 원점수 평균·표준편차.

평가지표 정의 (3종류를 모두 계산하는 이유)

(1) Interval F1 (겹침 기반, point-adjustment 아님)

TP_pred = 예측 구간 중 GT 구간과 하나라도 겹치는 것의 개수

TP_gt = GT 구간 중 예측 구간과 하나라도 겹치는 것의 개수

FP = 안 겹치는 예측 구간 수, FN = 안 겹치는 GT 구간 수

$P = TP_pred / (TP_pred + FP)$, $R = TP_gt / (TP_gt + FN)$, $F1 = 2PR / (P + R)$

-> 원본 Stage2(단변량) 파이프라인이 쓰는 지표와 동일 (경계가 조금 달라도 겹치기만 하면 정답)

(2) Point-wise F1 (point-adjustment 없음, 이번 세션 SMD 트랙 전체에서 계속 쓰던 방식)

최종 예측 구간들을 시점별 이진 배열로 변환 후, 매 시점 단위로 정직하게 TP/FP/FN 계산.

-> 경계가 부정확하면(구간이 좁아지거나 밀리면) 바로 점수 하락 -> interval F1보다 훨씬 엄격.

(3) PA F1 (Point-Adjustment, AnomalyTransformer/DCdetector 등 SOTA 논문 관행)

GT 구간 내부에 예측 점이 하나라도 있으면 그 구간 전체를 맞춘 것으로 간주(1로 채움) 후 (2)와 동일하게 point-wise F1 계산.

-> 셋 다 보고하는 이유: (1)은 원본 Stage2와 비교 가능, (2)는 이번 세션 전체 결과와 비교

가능, (3)은 SMD SOTA 논문들(OmniAnomaly, AnomalyTransformer 등)과 비교 가능. 특히 (3)은 '무작위 점수조차 SOTA를 이길 수 있다'는 비판(Kim et al., AAAI 2022)이 있어 (2)를 주 지표로,

(3)은 참고용으로만 병기.

TimeRCD 단독 baseline: best_prf() -- 300개 threshold를 percentile 50~99.9 사이에서 스윕해 point-wise F1이 최대가 되는 지점을 채택 (이번 세션 내내 쓰던 방식과 동일, threshold 튜닝 없이 test set에서 직접 최적점을 찾는 oracle-threshold 성격의 upper-bound 수치).

현재까지 결과 (진행 중 스냅샷)

(PDF 생성 시점까지 완료된 entity만 표시 -- 파일럿 실행 중이면 일부만 나올 수 있음)

entity	TimeRCD	단독	S1 (interval)	+S2(interval)	S1 (point)	+S2(point)	S1 (PA)	+S2(PA)

machine-2-2	0.3657		0.4000	0.5098	0.3740	0.3358	0.4202	0.3358
machine-2-9	0.3430		0.3025	0.3564	0.2557	0.2013	0.3346	0.2553
machine-3-1	0.6940		0.1739	0.2759	0.0372	0.0320	0.0372	0.0320